2024 IEEE 2nd International Conference on Artificial Intelligence, Blockchain, and Internet of Things (AIBThings) | 979-8-3315-2952-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/AIBThings63359.2024.10863546

Optimizing Resource Utilization and Power Efficiency in FPGA-Accelerated YOLOv8 Object Detection Using Vivado High-Level Synthesis (HLS) Tool

Essel Emmanuel
Electrical Engineering Department
Southern University and A & M College
Baton Rouge, USA
emmanuel.essel@sus.edu

Fred Lacy
Electrical Engineering Department
Southern University and A & M College
Baton Rouge, USA
fred_lacy@subr.edu

Yasser Ismail
Electrical Engineering Department
Southern University and A & M College
Baton Rouge, USA
yasser_ismail@subr.edu

***Abstract*— Convolutional Neural Networks (CNNs) are crucial in computer vision due to their high accuracy in object detection tasks, but their processing demands necessitate efficient hardware acceleration solutions. FPGAs are ideal for this purpose due to their reconfigurability, parallelism, and low power consumption. This research addresses the challenges of high resource utilization and power consumption in existing YOLOv8 FPGA hardware accelerators by proposing an optimized approach using the Vivado High-Level Synthesis (HLS) tool. Unlike a prior study that reported high resource usage, our method reduces resource utilization by 9.2%. The YOLOv8 model is converted from PyTorch to the ONNX format for better visualization and extraction of layer configurations and parameters for FPGA implementation. Utilizing Xilinx Vivado HLS and the Vivado environment, the YOLOv8 architecture is designed for the ZYNQ-7 ZC706 board. By employing a 16-bit fixed-point representation, our approach improves resource utilization efficiency. A significant reduction in power usage to 9.342W, outperforming previous works even with a virtual FPGA board, is achieved after performing synthesis and implementation on the design.**

Keywords— FPGA Acceleration, YOLOv8, Vivado High-level Synthesis (HLS), Resource Utilization, Power Efficiency

I. INTRODUCTION

In recent years, embedded systems have integrated deep learning models with growing interest, driven by the demand for real-time and low-power solutions across a wide range of applications. Among such applications, object detection is one of the fundamental tasks that have gained vast interest, ranging from surveillance and autonomous vehicles to industrial automation and augmented reality. The development of object detection algorithms witnessed the establishment of the You Only Look Once (YOLO) series, characterized by real-time performance with high accuracy [1].

The YOLO family of models, by reframing object detection as a single regression problem, lets the predictions be directly made straight from the full image within one pass through the network. This approach contrasts with traditional methods that rely on sliding window approaches or region proposal networks (RPN), which results in much faster inference times without loss of accuracy. The YOLO models especially versions such as YOLOv3 and its successors, have demonstrated impressive performance on benchmark datasets such as COCO (Common Objects in Context), achieving state-of-the-art results in terms of both speed and accuracy [2].

Integration of the YOLO models into FPGA devices is a natural progression in the quest for efficient and scalable solutions for object detection within resource-constrained environments [3]. FPGAs allow for programmable hardware acceleration, which can directly map complex neural network architectures onto the chip. This integration promises real-time inference ability while adhering to stringent power and resource constraints.

Also, in the context of FPGA acceleration for Convolutional Neural Networks (CNNs), the Xilinx Vivado High-Level Synthesis (HLS) and Vivado Design Suite have become indispensable tools [4]. Vivado HLS enables the description of algorithms and functions in high-level languages like C, C++, or SystemC, which can then be synthesized into RTL code suitable for FPGA implementation [5]. This high-level abstraction in a sense enables fast prototyping and optimization of CNN architectures that ease the development process.

II. EXISTING WORKS

Resource and power efficiency play a vital role in FPGA accelerators for YOLO models. The project addresses the challenge of high resource utilization and power consumption in FPGA-accelerated YOLOv8 object detection models, as demonstrated by [6]. In the SATAY (A Streaming Architecture Toolflow for Accelerating YOLO Models) approach, the

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on September 30,2026 at 07:58:42 UTC from IEEE Xplore. Restrictions apply.

authors developed a streaming architecture toolflow designed specifically to accelerate YOLO models on FPGA devices. Their solution involved creating a deeply pipelined, on-chip implementation of the entire YOLO model, leveraging a streaming architecture that could target a wide range of compatible FPGA devices. This architecture was generated using an automated toolflow, allowing for the creation of accelerators for various YOLO models, including YOLOv3-tiny, YOLOv5, and YOLOv8. A significant achievement of this work was the first implementation of YOLOv8 on FPGA, demonstrating the adaptability and efficiency of their toolflow. However, limitations were encountered in achieving optimal resource utilization and power efficiency.

Other works tackled the issue of high resource utilization and power consumption in FPGA-accelerated object detection models through different methods, but each encountered limitations in achieving optimal efficiency. [7] focused on a hardware accelerator for the YOLOv4-tiny model, enhancing real-time object detection in self-driving cars using parallel processing and pipeline architecture, but it was limited by resource constraints, memory bandwidth, and scalability issues. [8] proposed a configurable hardware accelerator for YOLO, optimizing resource utilization and power efficiency through a modular design and strategic memory management, but it struggled with complexity, scalability, and real-time performance. Finally, an approach using high-level synthesis (HLS) tools to deploy the YOLOv7-tiny model on FPGA devices successfully reduced resource usage and power consumption, though it faced accuracy trade-offs, FPGA resource constraints, and optimization complexity [16]. Despite these challenges, the approach represented a significant advancement in improving the efficiency of YOLOv7-tiny on FPGA platforms.

To address previous limitations, the proposed plan uses the parameterizable FPGA-tailored architecture from [9], which adapts to various YOLO models to optimize resource usage and performance. The design allows customization of processing elements, buffer sizes, and data paths to match various FPGA resources and application needs. Techniques like pipelining, parallelism, and channel interleaving enhance throughput and reduce latency, while efficient memory access and parameter quantization minimize resource usage and power consumption. This results in a scalable, efficient system for FPGA-accelerated YOLOv8 object detection, addressing challenges related to high resource utilization and power consumption.

This paper presents the implementation of FPGA-accelerated YOLOv8 for object detection, focusing on reducing resource utilization and power consumption. The approach involves optimizing each layer of the YOLOv8 model using high-level synthesis tools, efficient data handling via DMA, and quantization techniques. By adapting the YOLOv8 architecture to an FPGA environment with Vivado HLS, and employing techniques like fixed-point representation, the study achieves significant reductions in resource usage and power consumption, making it a suitable solution for energy-efficient, high-performance scenarios. The rest of this paper is organized as follows. The proposed methodology is described and explained in Section III. Section IV illustrates the comparisons and discusses the results. Finally, the paper concludes in Section V.

III. PROPOSED METHODOLOGY

First, the official YOLOv8 pre-trained model and Python are used to train the network. The trained model's properties are all contained in the output model file, which is a .pt file (PyTorch model extension file). Next, the .pt output file is further exported to an ONNX file which makes it much easier to view the network properties and data in a graphical representation form depicted in Fig. 1. In viewing the network, Netron which is a viewer for neural networks, deep learning, and machine learning models is used. Table 1 shows the number of layers analyzed from the model.

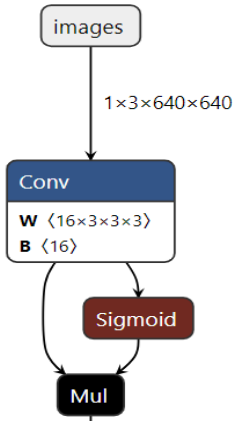


Figure 1: A chunk of the YOLOv8 ONNX model.

Table 1: NUMBER OF LAYERS IN YOLOV8.

LAYERS	COUNTS
1 x 1 Conv	25
3 x 3 Conv	38
Upsample	2
Max pooling	5
Concatenation	19
Activation	56

A. DESIGN FLOW OF THE YOLOv8 ARCHITECTURE

In the design flow of my work, as depicted in Fig. 2, a Python code was initially written to generate each layer's information such as padding, dilation, stride, kernel size, weights, and biases. These parameters were used to implement each module in C++ in Vivado HLS. The target FPGA board chosen for this project was the ZYNQ-7 ZC706 Evaluation Board, which has sufficient resources for the implementation, including 362 I/O blocks, 218k LUTs, 437k flip-flops, 545 Block RAMs, and 900 DSPs. Initially, 32-bit floating-point values were used for all network data and weights, but due to high processing power and memory requirements, a 16-bit fixed-point representation was introduced using the ***ap_fixed.h*** library in Vivado HLS to quantize weights without affecting accuracy. Each module was designed in C++ and underwent C synthesis and C simulation to determine resource utilization and error checking. Line buffers and registers were used to store temporary results, optimizing latency. The Register Transfer Level (RTL) design was then exported to generate IP blocks for each module, which were further exported to the Vivado environment to implement the overall system architecture with other hardware components.

The system design process involved using the Vivado environment for synthesis and implementation to convert a high-level hardware description into a gate-level netlist and map it onto the target FPGA device. Optimizations were made for performance, area, and power consumption. The physical realization on the ZYNQ-7 ZC706 board involved placing and

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on September 30,2026 at 07:58:42 UTC from IEEE Xplore. Restrictions apply.

routing hardware components to ensure optimal layout and resource usage. Detailed reports provided performance metrics, and after successful implementation, a bitstream file was generated for programming the FPGA. The deployment and testing phase will verify functionality and evaluate performance metrics like latency, throughput, and power consumption, ensuring efficient deployment of the YOLOv8 architecture on the FPGA.

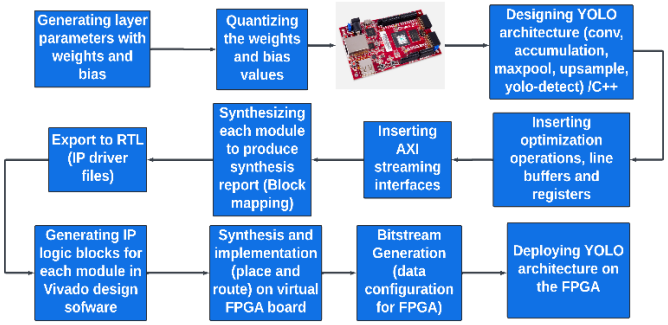


Figure 2: YOLOv8 Architecture Design Implementation On FPGA.

B. YOLOv8 ARCHITECTURE DESIGN

In designing the YOLOv8 architecture, five modules were designed in C++ which makes up the complete architecture. Also, pragma directives such as LOOP PIPELINING, LOOP FLATTENING, and LOOP unrolling were integrated into the FOR LOOPs in each module for optimization [10].

The FPGA-accelerated YOLOv8 Convolution module enhances performance and resource efficiency by implementing direct convolutions with a line buffer, optimized data transfer through channel interleaving, and parallelized inputs for faster operations. It uses a sliding window mechanism for data fetching, and memory efficiency is achieved by retaining only recent rows, despite added control logic complexity. Runtime adjustments of parameters such as channels, feature map dimensions, kernel size, and stride further optimize resource use and performance, improving the overall efficiency of the YOLOv8 implementation on FPGA. The Accumulation module in an FPGA-accelerated CNN handles resource limitations through channel folding. It divides convolution layers into smaller sub-layers when input or output channels exceed the FPGA's capacity, then accumulates their outputs. The module has two input streams, one output stream, and an AXI4-Lite port for configuration. It performs tasks like weighted input summation, bias addition, and activation functions using fixed-point multiplication. BRAM is used for bias storage, while DSP cores manage multiplication and control, optimizing resource utilization and performance. The Max Pooling module down-samples feature maps by selecting maximum values from non-overlapping regions, reducing computational complexity while preserving important features. The Up-sample module enhances spatial resolution through nearest-neighbor interpolation, reversing pooling effects to preserve details. It uses a line buffer for efficient data handling and specific parameters for effective upscaling, optimizing memory, and control logic for improved performance and accuracy. The YOLO-detect module divides images into grids for object detection, generating outputs such as bounding box dimensions, objectness scores, and class scores. It utilizes 80 channels for class scores, one for objectness, and four for bounding boxes. Sigmoid activation is applied selectively using AXI4-Lite, and Xilinx's fixed-point exponential function reduces errors with hardware switches.

C. SYSTEM LEVEL ARCHITECTURE

The FPGA-accelerated YOLOv8 architecture optimizes data and control flow among its components as shown in Fig. 3, including the processor system reset, AXI interconnect, and DMA modules for high-speed communication with off-chip DDR memory. DMA1 and DMA2 handle data transfers between the main memory and the system, while the processor IP manages coordination using AXI4-Lite to send parameters to the accelerator. Inside the accelerator, convolutional IP processes weights and input data through DMA, feeding into the accumulation & activation module for further operations like folding, biasing, and activation. Outputs then move to stages such as max pooling, yolo-detect, or up-sample IPs based on the network typology. The architecture, designed in Vivado block design, features three interconnected processing stages with AXI4-stream and switch IPs for flexible data routing. Switch 1 controls whether DMA 1 data goes directly into convolutional IP or another stage, while Switch 2 and Switch 3 manage post-accumulation processing steps such as max pooling, yolo-detect, up-sample, or no further processing at all. These switches allow for flexible adjustments to the processing pipeline, accommodating diverse architectural needs.

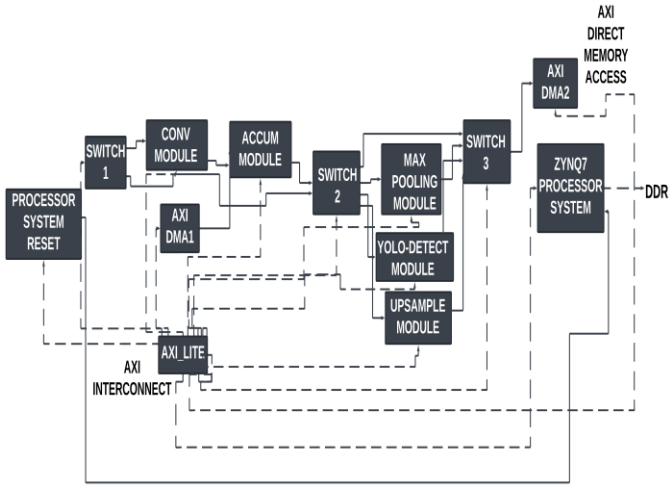


Figure: 3 System-level Architecture.

IV. RESULTS AND DISCUSSION

The study compares the performance of a hardware-accelerated YOLOv8 algorithm on Xilinx FPGA with other works, focusing on resource utilization and power consumption. Implemented on a virtual ZYNQ-7 ZC706 using Vivado-HLS, the design achieved 58.4K look-up tables, 78K flip-flops, 306.5 BRAMs, and 196 DSPs with 16-bit precision. The synthesis met real-time object detection requirements with a power efficiency of approximately 9.342 W. Also, Table 2 shows that every SoC, except for [11] and [12], has an

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on September 30,2026 at 07:58:42 UTC from IEEE Xplore. Restrictions apply.

Table 2 Object detection results on FPGAs

Platform	Network	LUT	FF	BRAM	DSP	Power (W)	OTHER WORKS
Zynq-7035	FPGA YOLO	47k	40k	787	409	7.518	[14]
ZCU102	Lightweight YOLOv2	135k	370k	1706	377	4.5	[15]
VC707	Sim-YOLOv2	155k	115k	1144	272	18.29	[13]
Zed board	YOLOv3_tiny	25.9k	46.7k	185	160	3.36	[9]
XC7Z7045	Angel-Eye	182.6k	127.6k	486	780	9.63	[16]
XC7Z7020	YOLOv4	27k	24k	68	198	10.36	[11]
XC7Z7020	YOLOv4	23.2k	45.8k	115	174	N/A	[11]
XC7Z7100	YOLOv7-tiny	59k	9k	1092	16	10.79	[12]
XC7Z7045	YOLOv7-tiny	56k	10k	522	16	8.92	[12]
VCU118	YOLOv3-Tiny	431K	-	2148	6687	42.9	[6]
VCU118	YOLOv5	1170K	-	2026	5077	67.0	[6]
VCU110	YOLOv8	629K	-	2782.5	1767	57.4	[6]
ZYNQ-7 ZC706	YOLOv8	58.4K	78K	306.5	196	9.342	This work

equivalent or higher number of resources. Though their network is less complex than ours, [13] proposes a lightweight implementation of the YOLOv2 network on an FPGA with the ZCU102 board, which is significantly more expensive than ours.

But compared to our accelerator, both of the accelerators in [13], and [15] demand greater resource utilization. Our suggested method for YOLOv8 on the ZYNQ-7 ZC706 with a much more complex network structure and a large number of parameters can be implemented for real-time applications on low-budget SoCs. This is because scaled versions of YOLOv2 and YOLOv3 in [15]and [9] have fewer parameters, and their corresponding resource utilization reflects this issue. Adib et al [12] employed HLS for accelerator design evaluation on a low-budget SoC such as XC7Z7045. By using the Vitis-HLS tool for synthesis and implementation of the YOLOv7-tiny object detection algorithm, they achieved a resource utilization of 59 K LUT, 9 K Flip-flops, 1092 BRAMs, and 16 DSPs using 13-bit datatype precision. Their implementation achieved the second lowest resource utilization even though the YOLOv7-tiny has a complex network structure compared to [14], [15], [13], [9], and [16] in Table 2. The work [6] introduces SATAY (Streaming Architecture Toolflow for Accelerating YOLO), a toolflow created to implement advanced object detection models like YOLOv3, YOLOv5, and YOLOv8 on FPGA devices for real-time, low-latency edge-based detection. It is the first implementation of YOLOv8 on an FPGA. Using Vivado HLS, this approach reduced resource utilization by 9.2% and cut power consumption by 39% compared to other methods, achieving optimal power efficiency during synthesis and implementation as shown in Fig. 2

V. CONCLUSION

This research improves FPGA-based object detection using YOLOv8 by optimizing it in the Vivado HLS environment, achieving a 9.2% reduction in resource usage and a power consumption of 9.342W. Future work will test this design on a physical FPGA board to evaluate its real-world performance and validate the simulation results for practical applications in autonomous systems, smart surveillance, and advanced robotics.

REFERENCES

[1] Naif Alsharabi, "Real-Time Object Detection Overview: Advancements, Challenges, and Applications," *J. AMRAN Univ.*, vol. 3, no. 6, p. 12, 2023, doi: 10.59145/jaust.v3i6.73.

[2] J. Terven, D. M. Córdova-Esparza, and J. A. Romero-González, "A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Mach. Learn. Knowl. Extr.*, vol. 5, no. 4, pp. 1680–1716, 2023, doi: 10.3390/make5040083.

[3] E. Markov, Nikolay and Zoev, Ivan and Mytsko, *FPGA Hardware Implementation of the Yolo Subclass Convolutional Neural Network Model in Computer Vision Systems*. 2022.

[4] D. O'Loughlin, A. Coffey, F. Callaly, D. Lyons, and F. Morgan, "Xilinx vivado high level synthesis: Case studies," *IET Conf. Publ.*, vol. 2014, no. CP639, pp. 352–356, 2014, doi: 10.1049/cp.2014.0713.

[5] S. M. and S. B. and A. M. and A. C. A. and L. K. and M. Machhout, "Deep convolutional neural networks-based Hardware–Software on-chip system for computer vision application," *Comput. Electr. Eng.*, vol. 98, no. 0045–7906, p. 107671, 2022, doi: https://doi.org/10.1016/j.compeleceng.2021.107671.

[6] A. Montgomerie-Corcoran, P. Toupas, Z. Yu, and C. S. Bouganis, "SATAY: A Streaming Architecture Toolflow for Accelerating YOLO Models on FPGA Devices," *Proc. - Int. Conf. Field-Programmable Technol. ICFPT*, pp. 179–187, 2023, doi: 10.1109/ICFPT59805.2023.00025.

[7] Z. Valadanzoj, H. Daryanavard, and A. Harifi, "High-speed YOLOv4-tiny hardware accelerator for self-driving automotive," *J. Supercomput.*, vol. 80, no. 5, pp. 6699–6724, 2024, doi: 10.1007/s11227-023-05713-2.

[8] D. Pestana *et al.*, "A Full Featured Configurable Accelerator for Object Detection with YOLO," *IEEE Access*, vol. 9, pp. 75864–75877, 2021, doi: 10.1109/ACCESS.2021.3081818.

[9] Z. Yu and C. S. Bouganis, *A Parameterisable FPGA-Tailored Architecture for YOLOv3-Tiny*, vol. 12083 LNCS. Springer International Publishing, 2020.

[10] J. Zhao, L. Feng, S. Sinha, W. Zhang, Y. Liang, and B. He, "COMBA: A comprehensive model-based analysis framework for high level synthesis of real applications," *IEEE/ACM Int. Conf. Comput. Des. Dig. Tech. Pap. ICCAD*, vol. 2017-Novem, pp. 430–437, 2017, doi: 10.1109/ICCAD.2017.8203809.

[11] P. Babu and E. Parthasarathy, "Hardware acceleration for object detection using YOLOv4 algorithm on Xilinx Zynq platform," *J. Real-Time Image Process.*, vol. 19, no. 5, pp. 931–940, 2022, doi: 10.1007/s11554-022-01234-y.

[12] A. Hosseiny and H. Jahanirad, "Hardware acceleration of YOLOv7-tiny using high-level synthesis tools," *J. Real-Time Image Process.*, vol. 20, no. 4, pp. 1–10, 2023, doi: 10.1007/s11554-023-01324-5.

[13] Y. J. Wai, Z. bin M. Yussof, S. I. bin Salim, and L. K. Chuan, "Fixed point implementation of Tiny-Yolo-v2 using OpenCL on FPGA," *Int. J. Adv. Comput. Sci. Appl.*, vol. 9, no. 10, pp. 506–512, 2018, doi: 10.14569/IJACSA.2018.091062.

[14] G. Wei, Y. Hou, Q. Cui, G. Deng, X. Tao, and Y. Yao, "YOLO Acceleration using FPGA Architecture," *2018 IEEE/CIC Int. Conf. Commun. China, ICCC 2018*, no. Iccc, pp. 734–735, 2018, doi: 10.1109/ICCChina.2018.8641256.

[15] H. Nakahara, T. Fujii, H. Yonekawa, and S. Sato, "A lightweight YOLOv2: A binarized CNN with a parallel support vector regression for an FPGA," *FPGA 2018 - Proc. 2018 ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays*, vol. 2018-Febru, pp. 31–40, 2018, doi: 10.1145/3174243.3174266.

[16] K. Guo *et al.*, "Angel-eye: A complete design flow for mapping CNN onto customized hardware," *Proc. IEEE Comput. Soc. Annu. Symp. VLSI, ISVLSI*, vol. 2016-Septe, pp. 24–29, 2016, doi: 10.1109/ISVLSI.2016.129.

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on September 30,2026 at 07:58:42 UTC from IEEE Xplore. Restrictions apply.